V.E.S.P.E.R.

Dokumentacja Techniczno-Ruchowa (DTR)

Opracowanie techniczne w formie raportu naukowo-inzynierskiego

Wersja: 1.2 (FULL - rozszerzona)
Data: 2026-02-22

Pakiet zrodlowy	vesper_final_beta (Next.js 15 + Prisma/PostgreSQL + Puppeteer + OpenAI Tools)
Zakres	SocialBot, Identity Vault, GEO_INTEL oraz komponenty wspolne (ROI/Nova/Nymphora)
Charakter dokumentu	DTR + raport projektowy: cele, metodologia, architektura, E2E dataflow, utrzymanie
Klasyfikacja	Dokument wewnetrzny / wdrozeniowy; zawiera procedury bezpieczenstwa i retencji

Streszczenie (Abstract)

Niniejszy dokument przedstawia pelna dokumentacje techniczno-ruchowa systemu V.E.S.P.E.R. w ujeciu projektowym (raport naukowo-inzynierski). Opisuje architekture, moduly funkcjonalne, modele danych, interfejsy API oraz pelny przeplyw danych od komendy operatora do wynikow analizy i ich wizualizacji. Dokument zawiera rowniez rozdzialy dotyczace wymagan niefunkcjonalnych, bezpieczenstwa, zgodnosci prawnej, eksploatacji, diagnostyki oraz utrzymania. Celem jest zapewnienie kompletnego materialu wdrozeniowego (dla administratora) i analitycznego (dla operatora OSINT), umożliwiajacego powtarzalne uruchomienie i bezpieczna eksploatacje systemu.

Slowa kluczowe: OSINT, SocialBot, Identity Vault, GEO_INTEL, Puppeteer, Next.js, Prisma, PostgreSQL, AI analyst, data provenance.

Spis tresci

Spis tresci w tej wersji jest wprowadzony jako lista. W MS Word/LibreOffice mozna wstawic automatyczny spis tresci na podstawie naglowkow.

- 1. Wstep i cele projektu
- 2. Zakres funkcjonalny i przypadki uzycia
- 3. Metodologia projektowa i zalozenia
- 4. Bezpieczenstwo, zgodnosc prawna i ostrzezenia
- 5. Architektura (warstwy, komponenty, interakcje)
- 6. Moduly systemu - opis pelny (frontend, backend, scrapers, AI, vault, geo, breach providers)
- 7. Przeplyw danych end-to-end (DFD/sequence)
- 8. Modele danych i schemat bazy (Prisma/PostgreSQL)
- 9. Interfejsy API i kontrakty (route handlers)
- 10. Konfiguracja srodowiska i parametryzacja (.env)
- 11. Instalacja, uruchomienie i wdrozenie
- 12. Eksploatacja (procedury operatora)
- 13. Diagnostyka i rozwiazywanie problemow
- 14. Wymagania niefunkcjonalne (wydajnosc, niezawodnosc, observability)
- 15. Polityka danych (retencja, backup, klasyfikacja informacji)
- 16. Testy, walidacja i kryteria akceptacji
- 17. Ograniczenia i kierunki rozwoju
- 18. Zalaczniki (drzewo projektu, pseudokod, formaty JSON, checklisty)
- 19. Zakonczenie

1. Wstep i cele projektu

V.E.S.P.E.R. jest systemem analitycznym wspierajacym prace OSINT/Threat Intelligence. Projekt zostal zaprojektowany jako aplikacja webowa typu full-stack, w ktorej warstwa prezentacji udostepnia operatorowi interfejs w stylu terminala oraz panele analityczne. Warstwa serwerowa orchestruje procesy pozyskania danych (scraping), analizy wspomaganej AI, utrwalania danych w bazie i na dysku, wzbogacania tozsamosci na podstawie danych z wyciekow oraz wizualizacji geograficznej.

1.1 Motywacja i problem badawczy

W praktyce OSINT kluczowe znaczenie ma laczenie danych rozproszonych: profilu w mediach spolecznosciowych, sladow sieciowych (identyfikatory, hinty lokalizacyjne), historii publikacji oraz informacji o kompromitacjach (wycieki). Problemem jest powtarzalnosc procesu oraz jego udokumentowana wiarygodnosc (data provenance). V.E.S.P.E.R. rozwiazuje ten problem przez: (a) automatyzacje pozyskania danych z zachowaniem surowych zrzutow JSON jako materialu dowodowego, (b) ujednolicenie struktury wynikow, (c) systematyczne wzbogacanie profilu o dane z wyciekow oraz (d) wizualizacje osi czasu i zdarzen geograficznych.

1.2 Cele glowne i mierzalne rezultaty

1. Zapewnienie powtarzalnego pipeline E2E: komenda -> skan -> analiza -> zapis -> prezentacja.
2. Utrwalenie danych z procesu (snapshoty + rawJsonData + pliki w public/scrapes).
3. Wprowadzenie warstwy Identity Vault z procedura promocji (promote) i kontrola danych zweryfikowanych.
4. Wprowadzenie GEO_INTEL: ekstrakcja i geokodowanie lokalizacji oraz ich prezentacja na mapie 3D.
5. Udokumentowanie mechanizmow bezpieczenstwa, retencji, backupu i odpowiedzialnego uzycia narzedzia.

2. Zakres funkcjonalny i przypadki uzycia

2.1 Uzytkownicy i role

- Operator OSINT: wykonuje skany, interpretuje wyniki, podejmuje decyzje o promocji danych z wyciekow.
- Analityk/SOC: wykorzystuje dane do triage, korelacji z innymi sygnalami i raportowania.
- Administrator: wdraza system, zarzadza baza, sekretami, backupami i polityka retencji.

2.2 Podstawowe przypadki uzycia (UC)

UC-1: Skan profilu (quick/deep)

Operator wskazuje target (username) i inicjuje skan w trybie quick lub deep. System uruchamia scraper, pozyskuje dane profilu i wybranych postow, generuje analize AI, zapisuje snapshot oraz zwraca wynik do UI. Tryb deep zwieksza zakres analizy (np. wiecej postow) oraz intensywnosc ekstrakcji metadanych.

UC-2: Wzbogacanie tozsamosci (Identity Vault)

Po skanie system automatycznie uruchamia zapytania do dostawcow informacji o wyciekach. Rekordy wyciekow sa zapisywane i wyswietlane w panelu IDENTITY_VAULT. Operator moze wykonac reczne wyszukiwanie (manual enrichment) oraz 'promowac' wybrane dane do sekcji leakVerified* profilu.

UC-3: GEO_INTEL i analiza przestrzenna

System buduje liste zdarzen geoEvents, laczac lokalizacje z postow (locationName) oraz wskazowki z Ghost Data. Dla nazw miejsc wykonywane jest geokodowanie (OSM Nominatim) z cache. Wynik jest prezentowany na mapie 3D; operator moze kliknac zdarzenie i otworzyc media.

UC-4: Utrwalenie materialu i eksport

Kazdy skan tworzy snapshot w bazie oraz surowy zapis JSON w public/scrapes. Dodatkowo operator moze zapisac media do zewnetrznego Vault (SOCIALBOT_VAULT_PATH), co umozliwia archiwizacje dowodowa.

3. Metodologia projektowa i zalozenia

3.1 Podejscie warstwowe i separacja odpowiedzialnosci

Projekt zastosowal podejscie warstwowe: UI (React) jest odseparowane od warstwy API (Next.js route handlers). W warstwie API wyodrebniono: narzedzia (tools) wywolywane przez model, akcje (actions) stanowiace scenariusze biznesowe, serwisy (services) realizujace operacje domenowe, oraz silnik scrapingu (core/scraper-engine) odpowiedzialny za interakcje z przegladarka i ekstrakcje danych.

3.2 Zalozenia dowodowe i data provenance

Podstawowym zalozeniem bylo utrzymanie surowego materialu (rawJsonData, pliki w public/scrapes) w celu mozliwosci audytu i weryfikacji wnioskow analitycznych. AI w systemie jest traktowane jako komponent wspierajacy (assistive) - wyniki AI wymagaja interpretacji i weryfikacji przez operatora.

3.3 Zalozenia odpowiedzialnego uzycia

System zaprojektowano jako narzedzie defensywne/analityczne. Nie zawiera funkcji omijania zabezpieczen ani wykonywania dzialan ofensywnych. Operator ponosi odpowiedzialnosc za zgodnosc z prawem i regulaminami uslug, z ktorych pobierane sa dane.

4. Bezpieczenstwo, zgodnosc prawna i ostrzezenia

4.1 Zgodnosc prawna - wymagania minimalne

- Uzywaj wyłącznie w ramach uprawnien (zgoda, umowa, zlecenie, uzasadniony interes prawny).
- Uwzglednij RODO/GDPR: minimalizacja danych, cel przetwarzania, retencja, prawa podmiotow danych.
- Respektuj regulaminy serwisow zewnetrznych (ToS); w przypadku ryzyka naruszen stosuj tryby ograniczone (quick, limit postow).

4.2 Ryzyka techniczne i operacyjne

- Rate limiting / blokady: intensywny scraping moze skutkowac ograniczeniami; zalecane humanizacja (opoznienia) i limity.
- Bledy geokodowania: Nominatim zwraca hipotezy; nazwy miejsc niejednoznaczne moga wskazywac bledne koordynaty.
- Falszywe dopasowania w wyciekach: wyszukiwanie po username lub nazwie ma niska wiarygodnosc; wymagaj korelacji z innymi danymi.
- Wrazliwe dane (hasla): przechowywanie hasel wymaga restrykcyjnej polityki dostepu i retencji; w praktyce zalecane maskowanie lub usuwanie.

4.3 Bezpieczenstwo sekretow i infrastruktury

- Sekrety (OPENAI_API_KEY, klucze dostawcow wyciekow) przechowuj poza repozytorium: zmienne srodowiskowe lub manager sekretow.
- Ogranicz dostep sieciowy do aplikacji (VPN, reverse proxy z auth).
- Baza danych tylko w sieci prywatnej; stosuj osobne konto DB z minimalnymi uprawnieniami.
- Vault (dysk z mediami) szyfruj i kontroluj ACL; rozdziel srodowisko DEV/PROD.

5. Architektura (warstwy, komponenty, interakcje)

5.1 Widok warstwowy

```
Warstwa UI (React/Next.js Client Components)
 - Terminal (komendy), MainView (panele), SocialMap3D (mapa), Lightbox (media)
      |
      v
Warstwa API (Next.js Route Handlers)
 - /api (GET/POST streaming + tools), /api/promote-leak, /api/manage-verified, /api/save-media
      |
      v
Warstwa domenowa (Actions/Services)
 - VesperActions (scenariusze E2E)
 - Services: AI Analyst, StorageService, LeakService, Repozytoria DB, GeoMapper
      |
      v
Warstwa integracji (Scraper Engine / Providers)
 - Puppeteer + interceptory + extractory
 - Dostawcy breach intelligence (LeakCheck, LeakOsint, ...)
 - Nominatim geocoder
      |
      v
Warstwa danych (PostgreSQL + dysk)
 - Prisma models (targets, snapshots, breaches, locations)
 - public/scrapes (surowe JSON)
 - Vault (zewnetrzne media)
```

5.2 Interakcja z modelem AI (Tools)

System wykorzystuje mechanizm narzedzi (tools/function calling). Model otrzymuje liste funkcji (VESPER_TOOLS) i podejmuje decyzje, czy uruchomic akcje backendowe. Jest to podejscie hybrydowe: operator komunikuje sie z systemem w jezyku naturalnym, a model mapuje intencje na wywolania funkcji.

Narzędzie	Opis
scan_instagram_profile	Scanning Instagram profile. Session remains open after completion (allows further scanning).
manual_leak_search	Searches breach databases (Identity Vault) for a specific phrase (e.g., Full Name, old email) and assigns the results to an existing target (username). Use this when automatic scanning did not detect the name in the bio or when you have additional known data about the target.
end_investigation	FORCEFULLY closes the bot session, destroys the browser instance, and terminates operation.

5.3 Strumieniowanie odpowiedzi

POST /api zwraca odpowiedz w trybie strumieniowym (ReadableStream). Frontend odczytuje dane porcjami i interpretuje prefiksy. W aktualnej implementacji backend emituje przede wszystkim RESULT: oraz ERROR:.

Prefiks LOG: jest obslugiwany po stronie UI (zarezerwowany dla postepu), jednak moze nie byc wykorzystywany we wszystkich sciezkach.

Przyklady ramek:
RESULT:{"content":"...","data":{...target...}}
ERROR:{"message":"..."}

6. Moduly systemu - opis pelny

6.1 Warstwa prezentacji (Frontend UI)

6.1.1 app/page.tsx - klient strumieniowy i koordynator stanu

Komponent app/page.tsx pelni role centralnego koordynatora widoku. Utrzymuje liste celow (targets), stan wybranego celu (selectedTarget), oraz log terminalowy (chatLog). Interakcja z backendem odbywa sie przez POST /api z nastepujacymi elementami: messages (historia), selectedTarget (kontekst) oraz parametry dla zapisu mediow. Mechanizm streamingowy umozliwia natychmiastowe wyswietlanie czesciowych wynikow bez oczekiwania na zakonczenie skanowania.

- Mechanika: fetch -> response.body.getReader() -> dekodowanie chunkow.
- Rozpoznawanie prefiksow: LOG:/RESULT:/ERROR: (przetwarzanie linii/fragmentow).
- Live update: po RESULT zawierajacym data (target), UI aktualizuje liste celow bez pelnego odswiezenia.
- Refresh: po zakonczeniu operacji UI wywoluje GET /api, aby zsynchronizowac stan (w tym geoEvents).

6.1.2 Terminal - interfejs komend operatora

Terminal jest zrealizowany jako komponent UI przechowujacy i renderujacy wpisy czatu. Wpis operatora jest przesylany do /api. Wpisy systemowe (VESPER) reprezentuja odpowiedzi modelu oraz wyniki tool_calls. Mechanizm jest celowo zblizony do pracy w konsoli, co sprzyja powtarzalnosci dzialan i latwosci raportowania (kopiowanie logow).

6.1.3 MainView - panele analityczne

MainView grupuje funkcje analityczne w zakladkach. Jest to kluczowy element 'human-in-the-loop': operator moze porownywac wynik AI, surowe dane i wycieki, a nastepnie podejmowac decyzje o promocji danych lub ich usunieciu.

- PSYCH_PROFILE: interpretacja zachowan (AI) z sygnalami wejscia.
- IDENTITY_VAULT: lista wyciekow, promocja, kontrola danych zweryfikowanych.
- GEO_INTEL: mapa 3D (punkty i zdarzenia).
- TIMELINE: historia snapshotow (zmiany w czasie).
- POST_ANALYSIS: analiza postow (caption, daty, lokacje).
- COMM_VAULT: galeria mediów + zapis do Vault.
- RAW_DATA: surowy JSON + debug (audyt).

6.2 Warstwa API (Next.js Route Handlers)

6.2.1 /api (POST) - brama konwersacyjna i router narzedzi

Endpoint /api w trybie POST przyjmuje historie konwersacji (messages) i uruchamia model AI z lista narzedzi. Jezeli model zdecyduje o wywolaniu narzedzia (tool_call), backend mapuje je na funkcje domenowe (actions/services). Wynik operacji jest zwracany w strumieniu jako RESULT wraz z opcjonalnym obiektem data (target), co umozliwia natychmiastowa aktualizacje UI.

6.2.2 /api (GET) - synchronizacja stanu i pobranie celow

GET /api zwraca liste celow wraz z ostatnimi snapshotami. Dodatkowo serwer oblicza computedGeoEvents na podstawie rawJsonData (lub postsAnalysis) i dolacza je do odpowiedzi. Jest to kompromis: geokodowanie moze byc kosztowne, dlatego stosowany jest cache w GeoMapper.

6.2.3 Pozostale endpointy

- /api/promote-leak: awans danych z IdentityBreach do leakVerified* w SocialBotTarget oraz ustawienie risk=critical.
- /api/manage-verified: usuwanie danych zweryfikowanych z naglowka profilu (kontrola retencji i minimalizacji).
- /api/save-media: zapis wybranego obrazu do zewnetrznego Vault na dysku.

6.3 Warstwa domenowa (Actions/Services)

6.3.1 VesperActions - scenariusze biznesowe

VesperActions jest miejscem, w ktorym laczone sa kroki pipeline'u. Funkcja executeScan integruje: scraping, AI, storage, DB i identity enrichment. Funkcja manualLeakSearch pozwala operatorowi na dodatkowe zapytanie do dostawcow wyciekow. terminateSession zwalnia zasoby przegladarki.

6.3.2 StorageService - utrwalanie danych surowych

StorageService zapisuje wyniki skanu jako pliki JSON w public/scrapes/<username>/. Ten mechanizm jest kluczowy dla audytu: plik jest dowodem wejscia dla wszelkich wnioskow. W snapshotach przechowywana jest sciezka localPath oraz kopia rawJsonData (JSON w DB).

6.3.3 AiAnalyst - analiza wspomagana AI

AiAnalyst generuje profil behawioralny (psychological profile) w oparciu o cechy profilu i postow. Wynik jest tekstem uzytecznym dla analityka jako hipoteza. Zasada: AI nie jest zrodlem prawdy, lecz narzedziem wspierajacym. Dla powtarzalnosci zaleca sie zapisywanie promptow i wersji modelu w metadanych (kierunek rozwoju).

6.3.4 LeakService - Identity Vault

LeakService integruje wielu dostawcow. Kluczowe jest ujednolicenie struktury wynikow i kontrola wiarygodnosci (confidence). Dane wyszukiwane po email sa oznaczane jako HIGH, natomiast po username/nazwie jako LOW. Operator ma mozliwosc promocji danych do naglowka profilu (leakVerified*), co jest sygnalem zweryfikowanego ryzyka i moze podniesc priorytet sprawy.

6.4 Silnik scrapingu (Puppeteer) - zasada dzialania

Scraper jest odpowiedzialny za automatyczne otwieranie profilu i ekstrakcje danych. Dziala w sesji przegladarki, co pozwala na ponowne wykorzystanie kontekstu (cookies, cache) w kolejnych operacjach. Interceptor sieciowy zbiera dane uboczne (Ghost Data), ktore nie zawsze sa widoczne w DOM, np. identyfikatory i hinty lokalizacyjne. Następnie extractory pobieraja: BIO, avatar, liczniki, a w przypadku postow - caption, date, locationName i mediaUrl.

- Tryb quick: minimalny zakres, szybszy czas, mniejsze ryzyko blokady.
- Tryb deep: wiekszy zakres (wiecej postow/analiz), wiekszy koszt i ryzyko blokad.
- Humanizacja: opoznienia losowe (randomDelay) oraz unikanie nadmiernego odswiezania.

- Obsługa bledow: timeouty i fallbacki, wymagaja monitoringu w wdrozeniu produkcyjnym.

6.5 GEO_INTEL - zasada dzialania i interpretacja

GEO_INTEL laczy dane lokacyjne z roznych warstw: (a) locationName z postow (jawne lokacje), (b) Ghost Data (hinty z ruchu). Dla locationName wykonywane jest geokodowanie przez Nominatim oraz zapisywany jest cache w pamieci procesu. Wyniki mapy 3D sluza do eksploracji hipotez (np. obszary aktywnosci), jednak nie powinny byc traktowane jako dowod jednoznaczny bez korelacji.

6.6 Vault - magazyn mediow

Vault (SOCIALBOT_VAULT_PATH) stanowi zewnetrzny magazyn mediow zapisywanych manualnie przez operatora. Jego celem jest rozdzielenie danych dowodowych (obrazy) od danych aplikacyjnych (public/scrapes). Zalecane jest traktowanie Vault jako zasobu o wysokiej wrazliwosci: szyfrowanie, ACL, backupy, logowanie dostepu i procedury kasowania.

7. Przeplyw danych end-to-end (DFD/Sequence)

7.1 Flow A - Skan profilu (sequence)

Aktorzy: Operator, UI, API (/api), Model AI, VesperActions, Scraper, AI Analyst, Storage, DB, Leak Providers, GeoMapper

1) Operator -> UI: wpisanie komendy (np. "zeskanuj @user w trybie deep, 8 postow")
2) UI -> API: POST /api {messages:[...], context:{selectedTarget?}}
3) API -> Model AI: chat.completions(tools=VESPER_TOOLS)
4) Model AI -> API: tool_call scan_instagram_profile(username, mode, postLimit)
5) API -> VesperActions.executeScan()
  5.1 VesperActions -> Scraper.scrapeProfile()
    - BrowserSession start/attach
    - NetworkInterceptor zbiera Ghost Data
    - Extractor profilu: bio/avatar/stats
    - Extractor postow: caption/date/locationName/mediaUrl (limit)
    - mergeGeoData -> geoEvents
  5.2 VesperActions -> AiAnalyst.generatePsychologicalProfile() -> tekst analizy
  5.3 VesperActions -> StorageService.saveScanSession() -> public/scrapes/<user>/<ts>.json
  5.4 VesperActions -> TargetRepository.upsertTarget() -> SocialBotTarget + Snapshot + Location(s)
  5.5 VesperActions -> LeakService.searchBreaches() -> IdentityBreach (dedupe)
6) API -> UI (stream): RESULT:{content, data: enrichedTarget}
7) UI: aktualizacja widoku + GET /api (synchronizacja computedGeoEvents)

7.2 Flow B - Identity Vault (promote i manage)

Flow B obejmuje procesy, ktore maja charakter 'human-in-the-loop': operator decyduje, ktore dane z wycieku zostana uznane za relewantne i przeniesione do profilu.

Promote:
1) UI -> POST /api/promote-leak {targetId, breachId}
2) Backend: wybiera IdentityBreach, kopiuje pola do leakVerified*
3) Backend: ustawia risk='critical' oraz breach.isPromoted=true
4) UI: odswieza targety

Manage verified:
1) UI -> POST /api/manage-verified {targetId, type, value}
2) Backend: usuwa element z leakVerifiedEmails/Phones/Ips/Pass lub czyści dob/address
3) UI: odswieza target

7.3 Flow C - GEO_INTEL (z cache i geokodowaniem)

1) UI -> GET /api
2) Backend: dla kazdego targeta:
  - pobiera latestSnapshot.rawJsonData
  - jesli rawJsonData zawiera geoEvents -> zwraca bez zmian
  - w przeciwnym razie:
    a) wyciaga locationName z postsAnalysis
    b) dla każdej nazwy: GeoMapper.fetchCoordinates() -> Nominatim (z cache)

c) buduje geoEvents

3) UI: SocialMap3D renderuje eventy; klik -> Lightbox(mediaUrl)

8. Modele danych i schemat bazy (Prisma/PostgreSQL)

W tej sekcji przedstawiono modele danych z prisma/schema.prisma, ich role oraz relacje. Opis obejmuje moduly funkcjonalne wykorzystywane w V.E.S.P.E.R. oraz modele ogolne, ktore stanowia fundament dla pracy przypadkowej (ROI) i grafowej.

8.x Model: SocialBotTarget

SocialBot / Target - główny obiekt profilu (nagłówek i stan ryzyka).

Pole	Typ	Atrybuty	Opis
id	String	@id @default(cuid())	Identyfikator rekordu (cuid).
username	String	@unique	Id celu (zwykle instagram username), klucz unikalny.
fullName	String?		Imię i nazwisko/wyświetlana nazwa (jeśli dostępne).
instagramPk	String?	@unique	Id wewnętrzne profilu (pozyskane z Ghost Data), unikalne jeśli wystąpi.
isBusiness	Boolean	@default(false)	Czy profil jest biznesowy (heurystyka/ghost).
businessCategory	String?		Kategoria biznesowa (jeśli profil biznesowy).
publicEmail	String?		Email publiczny (np. z profilu biznesowego).
publicPhone	String?		Telefon publiczny (np. z profilu).
externalUrl	String?		Link zewnętrzny (np. landing page).
leakVerifiedEmails	String[]	@default([])	Zweryfikowane (promowane) emaile z IdentityBreach.
leakVerifiedPhones	String[]	@default([])	Zweryfikowane telefony z wycieków.
leakVerifiedPass	String[]	@default([])	Zweryfikowane hasła z wycieków (wrażliwe - retencja!).
leakVerifiedIps	String[]	@default([])	Zweryfikowane IP z wycieków.
leakVerifiedDob	String?		Zweryfikowana data urodzenia (pole pojedyncze).
leakVerifiedAddress	String?		Zweryfikowany adres (pole pojedyncze).
status	String	@default("active")	Status operacyjny (np. active).

risk	String	@default("unknown")	Klasyfikacja ryzyka (np. normal/critical) - ustawiana m.in. po promocji.
notes	String?		Notatki analityka/operatora.
createdAt	DateTime	@default(now())	Czas utworzenia.
updatedAt	DateTime	@updatedAt	Czas aktualizacji.
snapshots	SocialBotSnapshot[]		Relacja 1..n do SocialBotSnapshot.
locations	SocialBotLocation[]		Relacja 1..n do SocialBotLocation.
breaches	IdentityBreach[]		Relacja 1..n do IdentityBreach.

8.x Model: SocialBotSnapshot

SocialBot - historia skanów (materiał dowodowy + ścieżki do JSON).

Pole	Typ	Atrybuty	Opis
id	String	@id @default(cuid())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
targetId	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
target	SocialBotTarget	@relation(fields: [targetId], references: [id], onDelete: Cascade)	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
bio	String?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
followerCount	Int?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
followingCount	Int?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
localPath	String?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
rawJsonData	String?		Pole zgodne ze

			schematem Prisma; szczegoly interpretacji zalezne od modulu.
scrapedAt	DateTime	@default(now())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.

8.x Model: SocialBotLocation

SocialBot - lokacje powiązane z celem (ręczne/automatyczne).

Pole	Typ	Atrybuty	Opis
id	String	@id @default(cuid())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
targetId	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
target	SocialBotTarget	@relation(fields: [targetId], references: [id], onDelete: Cascade)	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
name	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
lat	Float?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
lng	Float?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
address	String?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
city	String?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
detectedAt	DateTime	@default(now())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.

8.x Model: IdentityBreach

Identity Vault - rekordy wycieków i ich promocja.

Pole	Typ	Atrybuty	Opis
id	String	@id @default(cuid())	Id rekordu wycieku (cuid).
targetId	String		FK do SocialBotTarget.
target	SocialBotTarget	@relation(fields: [targetId], references: [id], onDelete: Cascade)	Pole techniczne/relacyjne.
sourceName	String		Nazwa dostawcy/wycieku.
breachDate	String?		Data wycieku (jesli dostepna).
email	String?		Email z rekordu (jesli wystepuje).
username	String?		Username z rekordu (jesli wystepuje).
password	String?		Haslo (UWAGA: wrazliwe).
phone	String?		Telefon (jesli wystepuje).
ip	String?		Adres IP (jesli wystepuje).
dob	String?		Data urodzenia (jesli wystepuje).
address	String?		Adres (jesli wystepuje).
searchQuery	String		Pole techniczne/relacyjne.
confidence	String		Poziom wiarygodnosci dopasowania (HIGH/LOW).
isPromoted	Boolean	@default(false)	Czy dane zostaly awansowane do leakVerified* w target.
detectedAt	DateTime	@default(now())	Pole techniczne/relacyjne.

8.x Model: RoiCase

ROI - sprawa operacyjna (przypadek, priorytet, własność, status).

Pole	Typ	Atrybuty	Opis
id	String	@id @default(cuid())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.

caseId	String	@unique	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
title	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
description	String?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
category	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
priority	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
status	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
owner	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
team	String[]		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
assignees	String[]		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
createdAt	DateTime	@default(now())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
updatedAt	DateTime	@updatedAt	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
deadline	DateTime?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
dataType	String		Pole zgodne ze

			schematem Prisma; szczegoly interpretacji zalezne od modulu.
inputValue	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
sources	String[]		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
tags	String[]		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
trustLevel	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
tlp	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
relatedCases	String[]		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
relatedEntities	String[]		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
origin	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
mode	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
workflowTemplate	String?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
integrations	String[]		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
visibility	String		Pole zgodne ze schematem Prisma;

			szczegoly interpretacji zalezne od modulu.
accessUsers	String[]		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
caseToken	String?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
initialNote	String?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
files	CaseFile[]		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
notes	CaseNote[]		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
conversations	NovaConversation[]		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
novaMessages	NovaMessage[]		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.

8.x Model: Case

Core - ogólna sprawa (graph container).

Pole	Typ	Atrybuty	Opis
id	String	@id @default(cuid())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
type	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
title	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
description	String?		Pole zgodne ze

			schematem Prisma; szczegoly interpretacji zalezne od modulu.
createdAt	DateTime	@default(now())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
updatedAt	DateTime	@updatedAt	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
edges	Edge[]		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
entities	Entity[]		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.

8.x Model: Entity

Core - węzeł grafu (typ/wartość/props).

Pole	Typ	Atrybuty	Opis
id	String	@id @default(cuid())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
caseId	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
type	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
value	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
label	String?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
tags	Json?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.

props	Json?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
createdAt	DateTime	@default(now())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
updatedAt	DateTime	@default(now())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
sources	Edge[]	@relation("Edge_source")	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
targets	Edge[]	@relation("Edge_target")	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
case	Case	@relation(fields: [caseId], references: [id], onDelete: Cascade)	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.

8.x Model: Edge

Core - relacja grafowa (source->target).

Pole	Typ	Atrybuty	Opis
id	String	@id @default(cuid())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
caseId	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
sourceId	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
targetId	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
type	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji

			zalezne od modulu.
label	String?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
createdAt	DateTime	@default(now())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
updatedAt	DateTime	@updatedAt	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
case	Case	@relation(fields: [caseId], references: [id], onDelete: Cascade)	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
source	Entity	@relation("Edge_source", fields: [sourceId], references: [id], onDelete: Cascade)	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
target	Entity	@relation("Edge_target", fields: [targetId], references: [id], onDelete: Cascade)	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.

8.x Model: NovaConversation

Nova - konwersacje analityczne (chat) z kontekstem JSON.

Pole	Typ	Atrybuty	Opis
id	String	@id @default(cuid())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
createdAt	DateTime	@default(now())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
updatedAt	DateTime	@updatedAt	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
title	String?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
caseId	String?		Pole zgodne ze schematem Prisma;

			szczegoly interpretacji zalezne od modulu.
case	RoiCase?	@relation(fields: [caseId], references: [id])	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
messages	NovaMessage[]		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.

8.x Model: NovaMessage

Nova - wiadomości konwersacji (role, content, toolSource).

Pole	Typ	Atrybuty	Opis
id	String	@id @default(cuid())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
createdAt	DateTime	@default(now())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
role	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
content	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
contextData	Json?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
toolSource	String?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
isPinned	Boolean	@default(false)	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
conversationId	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
caseId	String?		Pole zgodne ze

			schematem Prisma; szczegoly interpretacji zalezne od modulu.
attachments	NovaAttachment[]		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
case	RoiCase?	@relation(fields: [caseId], references: [id])	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
conversation	NovaConversation	@relation(fields: [conversationId], references: [id], onDelete: Cascade)	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.

8.x Model: NetworkHost

Nymphora - zasób sieciowy (host) do skanów.

Pole	Typ	Atrybuty	Opis
id	String	@id @default(cuid())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
target	String	@unique	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
type	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
createdAt	DateTime	@default(now())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
updatedAt	DateTime	@updatedAt	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
scans	NymphoraScan[]		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.

8.x Model: NymphoraScan

Nymphora - wynik skanu jako JSON.

Pole	Typ	Atrybuty	Opis
id	String	@id @default(cuid())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
target	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
scannedAt	DateTime	@default(now())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
data	Json		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
hostId	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
host	NetworkHost	@relation(fields: [hostId], references: [id], onDelete: Cascade)	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
roiCaseId	String?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.

8.x Model: StoredFile

Pliki - metadane plików (hash, path, klasyfikacja).

Pole	Typ	Atrybuty	Opis
id	String	@id @default(cuid())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
originalName	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
mimeType	String?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
size	Int?		Pole zgodne ze schematem Prisma;

			szczegoly interpretacji zalezne od modulu.
hash	String?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
checksum	String?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
diskPath	String		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
classification	String?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
source	String?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
threatScore	Int?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
isDeleted	Boolean	@default(false)	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
uploadedBy	String?		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
createdAt	DateTime	@default(now())	Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
caseFiles	CaseFile[]		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.
novaAttachments	NovaAttachment[]		Pole zgodne ze schematem Prisma; szczegoly interpretacji zalezne od modulu.

9. Interfejsy API i kontrakty (route handlers)

9.1 /api (GET)

- Cel: pobranie listy celow oraz ich ostatnich snapshotow.
- Wynik: lista obiektow targets z polami profilu + latestSnapshot + breaches + computedGeoEvents.
- Zastosowanie: synchronizacja UI po operacjach oraz inicjalny load strony.

9.2 /api (POST) - streaming + tools

- Cel: interfejs konwersacyjny; przetwarzanie komend w jezyku naturalnym i wywolanie narzedzi.
- Wejscie: JSON {messages:[{role,content}], selectedTarget?}.
- Wyjscie: text/plain stream z ramkami RESULT: / ERROR:.
- Uwagi: narzedzia sa opisane w VESPER_TOOLS i wybierane przez model (tool_choice=auto).

9.3 /api/promote-leak

- Wejscie: {targetId, breachId}
- Efekt: przeniesienie danych z IdentityBreach do leakVerified* w SocialBotTarget; risk ustawiany na 'critical'.
- Wyjscie: JSON {success, message}

9.4 /api/manage-verified

- Wejscie: {targetId, type, value} (type: email/phone/ip/pass/dob/address)
- Efekt: usuniecie/wyczyszczenie danych w leakVerified*
- Wyjscie: JSON {success, target}

9.5 /api/save-media

- Wejscie: {url, username}
- Efekt: zapis obrazu do Vault w folderze targets/<username>/<date>/
- Wyjscie: JSON {status, path, filename}

10. Konfiguracja srodowiska i parametryzacja (.env)

System jest parametryzowany przez zmienne srodowiskowe. Ze wzgledu na bezpieczenstwo sekrety nie powinny byc przechowywane w repozytorium. Ponizej przedstawiono zestaw minimalny oraz zalecenia produkcyjne.

10.1 Minimalny zestaw zmiennych

```
# Database
DATABASE_URL=postgresql://user:pass@host:5432/vesper

# OpenAI
OPENAI_API_KEY=...

# Identity Vault providers
LEAKCHECK_API_KEY=...
LEAKOSINT_API_KEY=...
SNUSBASE_API_KEY=...   # opcjonalnie

# Vault
SOCIALBOT_VAULT_PATH=/path/to/secure/vault
```

10.2 Zalecenia produkcyjne (hardening)

- Stosuj manager sekretow (Vault/KMS) zamiast pliku .env w systemie plikow.
- Rotuj klucze oraz ustaw limity i monitoring u dostawcow API.
- Uzywaj osobnych kluczy dla DEV i PROD.
- Stosuj zmienne dla kontroli logowania (np. LOG_LEVEL) i ogranicz logowanie wrazliwych danych.

11. Instalacja, uruchomienie i wdrozenie

11.1 Wymagania systemowe

- Node.js zgodny z Next.js 15 (zalecane LTS).
- PostgreSQL 13+ (zalecane 15+), stabilne polaczenie sieciowe.
- Srodowisko zdolne do uruchomienia Chromium (Puppeteer).
- Dysk na surowe zrzuty: public/scrapes oraz osobny dysk na Vault.

11.2 Instalacja (DEV)

```
npm install
npx prisma generate
npx prisma migrate dev
npm run dev
# http://localhost:3000
```

11.3 Wdrozenie (PROD)

```
npm ci
npx prisma generate
npx prisma migrate deploy
npm run build
npm run start
```

11.4 Konteneryzacja i CI/CD (rekomendacja)

Dla srodowisk produkcyjnych zaleca sie konteneryzacje. Puppeteer wymaga dodatkowych zaleznosci systemowych; Dockerfile powinien instalowac biblioteki dla Chromium. CI/CD powinno wykonywac testy, linter i migracje w kontrolowany sposob. Nalezy rowniez zabezpieczyc wolumen Vault i backup DB.

12. Eksploatacja (procedury operatora)

12.1 Procedura uruchomienia sesji analitycznej

6. Zweryfikuj, ze baza dziala oraz ze ustawione sa zmienne srodowiskowe (klucze API).
7. Uruchom aplikacje i wykonaj GET /api (UI powinno pobrac liste celow).
8. Dodaj target przez komende w terminalu (jezyk naturalny) lub wybierz istniejacy cel.
9. Wykonaj skan quick jako wstepny triage; w razie potrzeby wykonaj deep z limitem postow.
10. Przejrzyj RAW_DATA i porownaj z PSYCH_PROFILE (AI) oraz POST_ANALYSIS.
11. Sprawdz IDENTITY_VAULT: ocen confidence i korelacje; promuj dane tylko po weryfikacji.
12. W GEO_INTEL porownaj zdarzenia z osi czasu; oznacz hipotezy i notatki w profilu/sprawie.
13. Jesli zapisujesz media - uzyj COMM_VAULT -> Save do Vault; wprowadz zasady etykietowania.
14. Po sesji zakoncz investigation (zwolnij Puppeteer) i wykonaj backup (jesli wymagane).

12.2 Dobre praktyki OSINT i interpretacji

- Zawsze rozdziel dane surowe od wnioskow: w raporcie wskazuj, z ktorych artefaktow wynikaja wnioski.
- Dla danych z wyciekow stosuj co najmniej jedna korelacje niezalezna (np. email + nazwa + avatar).
- Dla GEO_INTEL sprawdzaj niejednoznacznosci nazw miejsc; w razie potrzeby weryfikuj manualnie.

13. Diagnostyka i rozwiazywanie problemow

13.1 Symptomy i przyczyny

- Brak wynikow skanu: target nie istnieje / brak dostepu / zmiana struktury serwisu.
- Timeout Puppeteer: zbyt agresywny limit czasowy, slabosc zasobow, blokady.
- Brak wynikow Identity Vault: brak kluczy, limity API, bledne zapytanie.
- Brak geoEvents: brak locationName w danych, brak dostepu do Nominatim, bledy geokodowania.
- Bledy DB: brak migracji, nieprawidlowy DATABASE_URL, brak uprawnien.

13.2 Procedura debugowania (checklist)

15. Sprawdz logi serwera (console) oraz odpowiedzi ERROR: w streamie.
16. Zweryfikuj polaczenie do bazy: npx prisma migrate status, a nastepnie prisma studio (opcjonalnie).
17. Uruchom testowy GET /api w przegladarce i sprawdz poprawny JSON.
18. Sprawdz konfiguracje Puppeteer w srodowisku (biblioteki, sandbox).
19. Zweryfikuj limity dostawcow breach intelligence (dashboard/API).
20. Weryfikuj foldery: public/scrapes oraz VAULT, uprawnienia zapisu.

14. Wymagania niefunkcjonalne (wydajnosc, niezawodnosc, observability)

14.1 Wydajnosc

Najbardziej kosztownymi operacjami sa: scraping (Puppeteer), geokodowanie (Nominatim) oraz wywolania AI. Wydajnosc zalezy od postLimit i trybu (quick/deep). Zalecenia: ograniczaj postLimit w triage, stosuj cache geokodowania, a dla AI rozdzielaj analize na porcje (kierunek rozwoju).

14.2 Niezawodnosc i odporność

- Retry z backoff dla dostawcow zewnetrznych (API, geokoder).
- Timeouty i circuit breaker dla scrapera.
- Idempotencja zapisu (deduplikacja breach records).
- Separacja danych: snapshot w DB + plik JSON na dysku (dwie kopie).

14.3 Observability

- Wprowadzenie structured logging (JSON) i poziomow logowania.
- Metryki: czas skanu, liczba postow, liczba wynikow wyciekow, bledy dostawcow.
- Tracing: identyfikator zadania (scanId) propagowany przez logi, snapshot i wyniki.

15. Polityka danych (retencja, backup, klasyfikacja informacji)

15.1 Klasyfikacja

Publiczne	Dane jawne z profili i postow.
Wrazliwe	Dane z wyciekow (email, telefon, ip, haslo), identyfikatory wewnetrzne.
Dowodowe	Surowe JSON-y ze skanow, zrzuty mediow w Vault.

15.2 Retencja (rekomendacja)

- public/scrapes: 30-90 dni (zalezne od polityki organizacji).
- rawJsonData w DB: rozważyć skracanie lub kompresje, ewentualnie przeniesienie do obiektu (S3) i trzymanie referencji.
- Vault: retencja zgodna z procesem dowodowym (moze byc dluzsza, ale z silnymi kontrolami dostepu).
- Hasla: preferowane maskowanie lub usuwanie po ekstrakcji sygnalu kompromitacji.

15.3 Backup

- DB: pg_dump dziennie/tygodniowo + szyfrowanie kopii.
- Vault: kopie przyrostowe + weryfikacja integralnosci (hash).
- Test odtworzenia: cykliczne proby restore w srodowisku testowym.

16. Testy, walidacja i kryteria akceptacji

16.1 Kryteria akceptacji (przyklad)

21. System uruchamia sie na czystym srodowisku zgodnie z instrukcja (npm, prisma, db).
22. GET /api zwraca liste celow i poprawne computedGeoEvents (gdy dane lokacyjne istnieja).
23. POST /api wywoluje scan_instagram_profile i zwraca RESULT zawierajacy data (target).
24. Po skanie w bazie istnieje nowy SocialBotSnapshot z localPath oraz rawJsonData.
25. IdentityBreach zapisywany jest z deduplikacja; promote-leak przenosi dane do leakVerified* i ustawia risk=critical.
26. save-media zapisuje plik w Vault w prawidlowej strukturze katalogow.

16.2 Testy jednostkowe i integracyjne (rekomendacja)

- Unit: LeakService (dedupe, mapowanie), GeoMapper (cache, normalizacja), repozytoria DB (upsert).
- Integration: /api (POST) z mock OpenAI; /api/promote-leak i /api/manage-verified z baza testowa.
- E2E: scenariusz skanu z kontrolowanym targetem (staging) i weryfikacja artefaktow (json, snapshot).

17. Ograniczenia i kierunki rozwoju

- Zmiany w serwisach zewnetrznych moga psuc scraping (konieczne utrzymanie extractorow).
- Brak pelnej standaryzacji dowodowej (np. podpisywanie snapshotow) - kierunek: hash, chain-of-custody.
- Cache geokodowania jest w pamieci procesu - kierunek: cache trwaly (DB/Redis) + rate limiting.
- AI: kierunek rozwoju to rejestrowanie promptow/wersji modelu i walidacja odpowiedzi (guardrails).
- Rozszerzenie integracji z ROI/Nova: przypinanie targetow do spraw, generowanie raportow, graf relacji.

18. Zalaczniki

18.1 Drzewo projektu (skrot)

shadowhunter@sara:~/vesper$ tree -I "node_modules"

```
.
├── app
│   ├── api
│   │   ├── manage-verified
│   │   │   └── route.ts
│   │   ├── promote-leak
│   │   │   └── route.ts
│   │   ├── route.ts
│   │   ├── save-media
│   │   │   └── route.ts
│   │   └── _vesper
│   │       ├── actions.ts
│   │       ├── geoMap.ts
│   │       ├── logger.ts
│   │       ├── services.ts
│   │       ├── tools.ts
│   │       └── types.ts
│   ├── favicon.ico
│   ├── globals.css
│   ├── layout.tsx
│   ├── page.tsx
│   └── types.ts
├── components
│   ├── Lightbox.tsx
│   ├── MainView.tsx
│   ├── profile-view
│   │   ├── tabs
│   │   │   ├── HistoryPanel.tsx
│   │   │   ├── IdentityPanel.tsx
│   │   │   ├── IntelPanel.tsx
│   │   │   ├── MediaPanel.tsx
│   │   │   ├── PostsPanel.tsx
│   │   │   ├── RawPanel.tsx
│   │   │   └── SocialMap3D.tsx
│   │   ├── TargetHeader.tsx
│   │   ├── vesper.png
│   │   └── WelcomeScreen.tsx
│   ├── ReportPDF.tsx
│   ├── TargetList.tsx
│   ├── Terminal.tsx
│   └── ui
│       ├── avatar.tsx
│       ├── badge.tsx
```

```
│       ├── button.tsx
│       ├── card.tsx
│       ├── dialog.tsx
│       ├── input.tsx
│       ├── scroll-area.tsx
│       ├── tabs.tsx
│       └── tooltip.tsx
├── components.json
├── core
│   ├── scraper-engine
│   │   ├── browser.ts
│   │   ├── extractors
│   │   │   ├── post.ts
│   │   │   └── profile.ts
│   │   ├── index.ts
│   │   ├── interceptor.ts
│   │   └── utils.ts
│   └── vault.ts
├── eslint.config.mjs
├── lib
│   ├── db.ts
│   └── utils.ts
├── next.config.ts
├── next-env.d.ts
├── package.json
├── package-lock.json
├── postcss.config.mjs
├── prisma
│   └── schema.prisma
├── public
│   ├── file.svg
│   ├── globe.svg
│   ├── next.svg
│   ├── vercel.svg
│   └── window.svg
├── README.md
├── tsconfig.json
└── types

17 directories, 63 files
```

18.2 Kontrakty danych - przykladowe struktury JSON

Przykladowy szkic rawJsonData (uproszczony):

```
{
  "profile": { "username": "...", "bio": "...", "avatarUrl": "...", "followers": 0, "following": 0 },
  "postsAnalysis": [
```

```
    { "postUrl": "...", "caption": "...", "date": "...", "locationName": "Warsaw", "mediaUrl": "..." }
  ],
  "ghostData": { "instagramPk": "...", "locations": [ ... ] },
  "geoEvents": [
    { "type":"post", "label":"Warsaw", "lat":52.23, "lng":21.01, "mediaUrl":"...", "date":"..." }
  ],
  "aiAnalysis": "tekst profilu psychologicznego"
}
```

18.3 Pseudokod pipeline executeScan

```
function executeScan(username, mode, postLimit):
  clean = normalize(username)
  raw = scrapeProfile(clean, mode, postLimit)
  analysis = AiAnalyst.generate(raw.profile.bio, raw.postsAnalysis)
  raw.aiAnalysis = analysis
  localPath = StorageService.save(raw)
  target = TargetRepository.upsert(clean, raw, localPath)
  breaches = LeakService.searchBreaches(target.publicEmail, target.username, target.fullName)
  LeakService.saveBreaches(target.id, breaches)
  return enrichTarget(target)
```

18.4 Checklist wdrozeniowy (admin)

27. Utworz baze PostgreSQL i konto z ograniczonymi uprawnieniami.
28. Skonfiguruj sekrety w managerze sekretow lub .env poza repo.
29. Zaplanuj wolumeny: public/scrapes oraz VAULT (szyfrowany).
30. Uruchom migracje Prisma (deploy).
31. Skonfiguruj reverse proxy i uwierzytelnianie dostepu do UI.
32. Skonfiguruj backup DB i backup VAULT + test restore.
33. Wlacz monitoring: logi, metryki, alerty limitow API.

19. Zakonczenie

Dokumentacja w wersji 1.2 stanowi rozbudowany opis projektu V.E.S.P.E.R. w formie raportu naukowo-inzynierskiego oraz DTR. Zawiera kompletne omowienie modulow, pelne przeplywy danych, modele bazy, interfejsy API i procedury eksploatacyjne. Dalszy rozwoj powinien koncentrowac sie na standaryzacji dowodowej (hashing, chain-of-custody), rozszerzeniu cache i limitowania zewnetrznych uslug, testach automatycznych oraz integracji modulow spraw (ROI/Nova) z analiza SocialBot.

Rysunki (zalacznik)

Rysunek 1. Podglad stron dokumentacji / diagramow (material pomocniczy).

V.E.S.P.E.R.

Dokumentacja techniczno-ruchowa (DTR) / Dokumentacja systemowa

Virtual Entity for Social Profiling & Evidence Retrieval

Wersja dokumentu: 1.0

Wersja oprogramowania: vesper 0.1.0 (repozytorium: vesper_final_beta)

Data: 2026-02-22

Środowisko: Next.js 15 (App Router) + Prisma (PostgreSQL) + Puppeteer

Klauzula: POUFNE (materiały wewnętrzne).

Rysunek 2. Podglad stron dokumentacji / diagramow (material pomocniczy).

Spis treści

1. Wstęp

2. Zakres, definicje i skróty

3. Opis systemu i architektury

4. Wymagania systemowe

5. Instalacja i uruchomienie

6. Konfiguracja (.env, cookies, magazyn danych)

7. Moduły systemu (opis szczegółowy)

8. Przepływy działania (use-cases)

9. Interfejs użytkownika i komendy

10. Baza danych (model i relacje)

11. Bezpieczeństwo, zgodność prawna i ostrzeżenia

12. Utrzymanie, kopie zapasowe i monitorowanie

13. Rozwiązywanie problemów (Troubleshooting)

14. Załączniki (szablony, endpointy, struktura katalogów)

15. Zakończenie

Rysunek 3. Podglad stron dokumentacji / diagramow (material pomocniczy).

V.E.S.P.E.R.

Dokumentacja Techniczno-Ruchowa (DTR)

Wersja: 1.1 (FULL)

Data: 2026-02-22

Status: Dokumentacja wdrożeniowa / eksploatacyjna

Nazwa systemu	V.E.S.P.E.R. (OSINT / SocialBot / Identity Vault / GEO_INTEL)
Technologie	Next.js 15 (App Router), TypeScript, Prisma, PostgreSQL, Puppeteer, OpenAI Tools, DeckGL/Map, OSM Nominatim
Pakiet źródłowy	vesper_final_beta
Cel dokumentu	Pełny opis modułów, przepływów danych, API i utrzymania